

DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

TOPIC 01

Experiment No: 1

Experiment Name

Finding the First Palindromic String in an Array

Aim

To develop a Python program that identifies and returns the first palindromic string from a given array of strings. If no palindrome exists, the program returns an empty string ("").

Algorithm

Step 1: Start the program.

Step 2: Read the list of strings from the user.

Step 3: Traverse each string in the array one by one.

Step 4: Reverse the current string.

Step 5: Compare the original string with its reversed version.

Step 6: If both are equal, print the string as the first palindrome and stop.

Step 7: Otherwise, continue checking the next string.

Step 8: If no palindrome is found after checking all strings, print an empty string ("").

Step 9: Stop the program.

Source code



```
main.py
1 def first_palindrome(words):
2     for word in words:
3         if word == word[::-1]:
4             return word
5     return ""
6 words = ["abc", "car", "ada", "racecar", "cool"]
7 result = first_palindrome(words)
8 if result:
9     print("First Palindromic String:", result)
10 else:
11     print("No Palindromic String Found")
```

Output

```
Output
First Palindromic String: ada

=== Code Execution Successful ===
```

Time Complexity

Let:

- n = Number of strings
- m = Average length of each string

Each string is checked once, and reversing a string takes $O(m)$ time.

Overall Time Complexity:

$$O(n \times m)$$

Space Complexity

The algorithm uses only a few extra variables.

Auxiliary Space Complexity:

$$O(1)$$

Result

Thus, the Python program to find the first palindromic string in an array was successfully implemented and executed.

Experiment No: 2

Experiment Name

Find the Number of Common Values Between Two Arrays

Aim

To develop a Python program that finds the number of common elements between two integer arrays and returns the counts as $[\text{answer1}, \text{answer2}]$, where:

- answer1 = Number of elements in nums1 that exist in nums2 .
- answer2 = Number of elements in nums2 that exist in nums1 .

Algorithm

Step 1: Start the program.

Step 2: Read the two integer arrays nums1 and nums2.

Step 3: Convert both arrays into sets for efficient searching.

Step 4: Initialize two counters:

- answer1 = 0
- answer2 = 0

Step 5: Traverse each element in nums1.

- If the element exists in nums2, increment answer1.

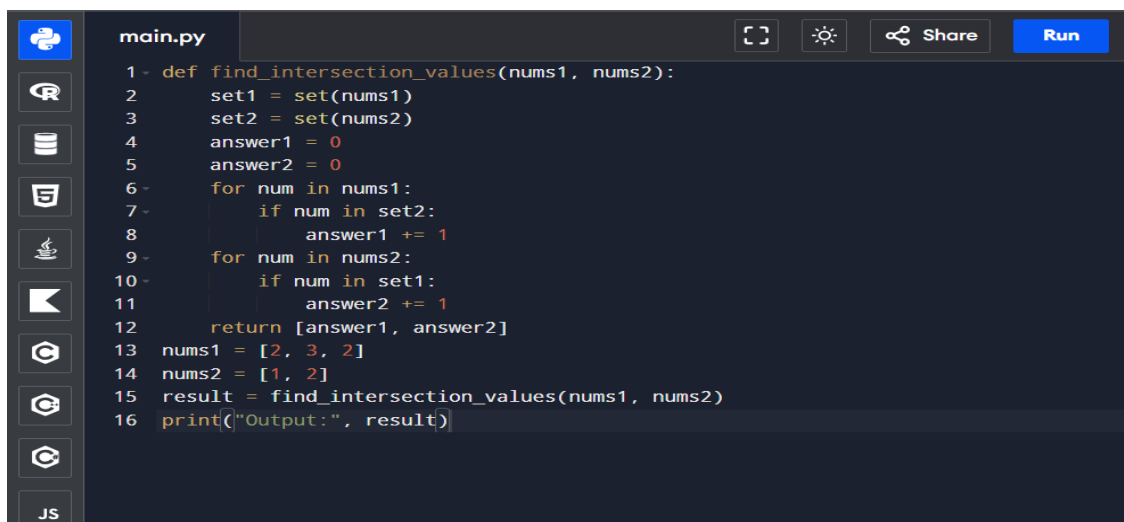
Step 6: Traverse each element in nums2.

- If the element exists in nums1, increment answer2.

Step 7: Display the result as [answer1, answer2].

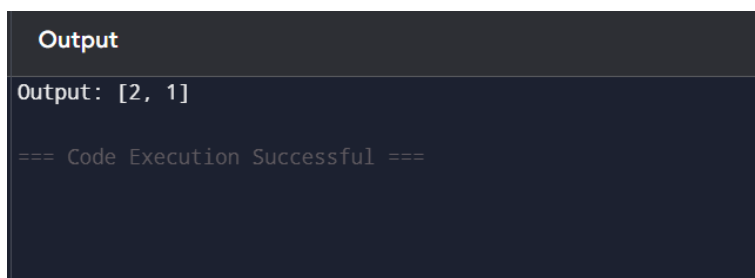
Step 8: Stop the program.

Source code



```
main.py
1- def find_intersection_values(nums1, nums2):
2-     set1 = set(nums1)
3-     set2 = set(nums2)
4-     answer1 = 0
5-     answer2 = 0
6-     for num in nums1:
7-         if num in set2:
8-             answer1 += 1
9-     for num in nums2:
10-        if num in set1:
11-            answer2 += 1
12-     return [answer1, answer2]
13- nums1 = [2, 3, 2]
14- nums2 = [1, 2]
15- result = find_intersection_values(nums1, nums2)
16- print("Output:", result)
```

Output



```
Output
Output: [2, 1]

=== Code Execution Successful ===
```

Time Complexity

Let:

- n = Size of nums1
- m = Size of nums2
- Creating sets $\rightarrow O(n + m)$
- Traversing both arrays $\rightarrow O(n + m)$

Overall Time Complexity:

$$O(n + m)$$

Space Complexity

Two sets are used to store array elements.

Space Complexity:

$$O(n + m)$$

Result

Thus, the Python program to find the number of common values between two arrays was successfully implemented and executed.

Experiment No: 3

Experiment Name

Sum of the Squares of Distinct Counts of All Subarrays

Aim

To develop a Python program that calculates the sum of the squares of the distinct element counts of all possible subarrays in a given integer array.

Algorithm

Step 1: Start the program.

Step 2: Read the integer array num.

Step 3: Initialize total_sum = 0.

Step 4: Traverse the array using index i as the starting point.

Step 5: Create an empty set to store distinct elements.

Step 6: Traverse the array using index j from i to the last element.

Step 7: Insert `num[j]` into the set.

Step 8: Find the number of distinct elements using the size of the set.

Step 9: Square the distinct count and add it to `total_sum`.

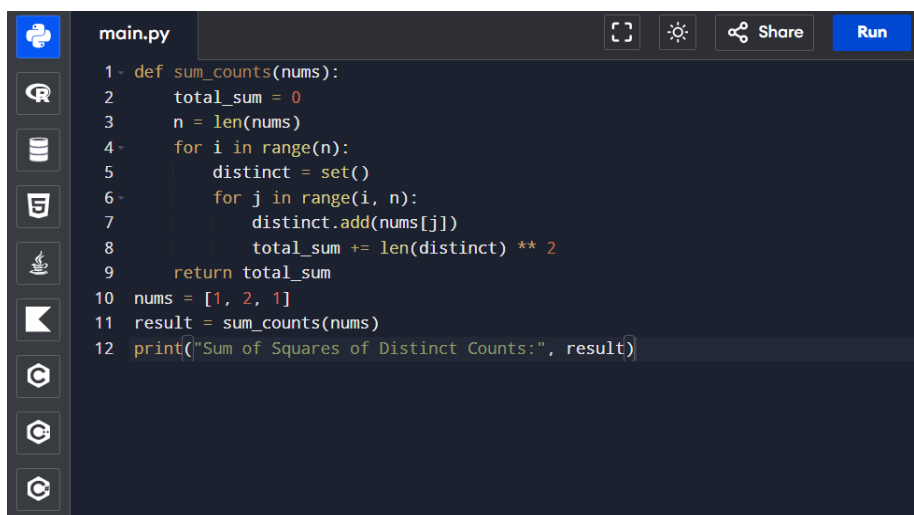
Step 10: Repeat Steps 6–9 until all subarrays starting from i are processed.

Step 11: Repeat Steps 4–10 for every starting index.

Step 12: Display the value of `total_sum`.

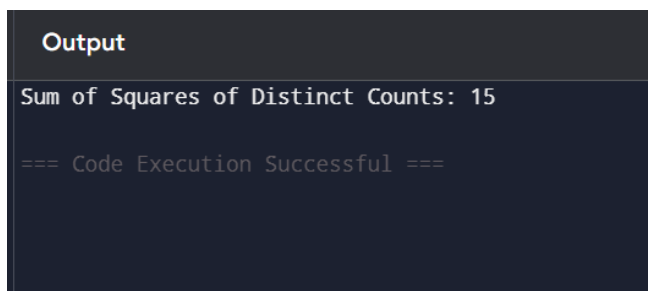
Step 13: Stop the program.

Source code

A screenshot of a code editor window titled 'main.py'. The editor has a dark theme and includes icons for file operations, a search icon, a settings icon, a 'Share' button, and a 'Run' button. The code is as follows:

```
1- def sum_counts(nums):
2-     total_sum = 0
3-     n = len(nums)
4-     for i in range(n):
5-         distinct = set()
6-         for j in range(i, n):
7-             distinct.add(nums[j])
8-             total_sum += len(distinct) ** 2
9-     return total_sum
10 nums = [1, 2, 1]
11 result = sum_counts(nums)
12 print("Sum of Squares of Distinct Counts:", result)
```

Output

A screenshot of an output window titled 'Output'. It displays the result of the code execution:

```
Sum of Squares of Distinct Counts: 15

=== Code Execution Successful ===
```

Time Complexity

Let n be the number of elements in the array.

- The outer loop executes n times.
- The inner loop executes approximately n times.
- Set insertion and size operations take $O(1)$ on average.

Overall Time Complexity

$$O(n^2)$$

Space Complexity

A set is used to store distinct elements of a subarray.

In the worst case, all elements are distinct.

Space Complexity:

$$O(n)$$

Result

Thus, the Python program to calculate the sum of the squares of the distinct counts of all subarrays was successfully implemented and executed.

Experiment No: 4

Experiment Name

Count Equal and Divisible Index Pairs in an Array

Aim

To develop a Python program that counts the number of pairs **(i, j)** in an array such that:

- $0 \leq i < j < n$
- $\text{nums}[i] == \text{nums}[j]$
- $(i \times j)$ is divisible by k

Algorithm

Step 1: Start the program.

Step 2: Read the integer array `nums` and integer `k`.

Step 3: Initialize `count = 0`.

Step 4: Traverse the array using index `i` from 0 to `n-1`.

Step 5: Traverse the array using index `j` from `i+1` to `n-1`.

Step 6: Check whether:

- `nums[i] == nums[j]`, and
- $(i \times j) \% k == 0$.

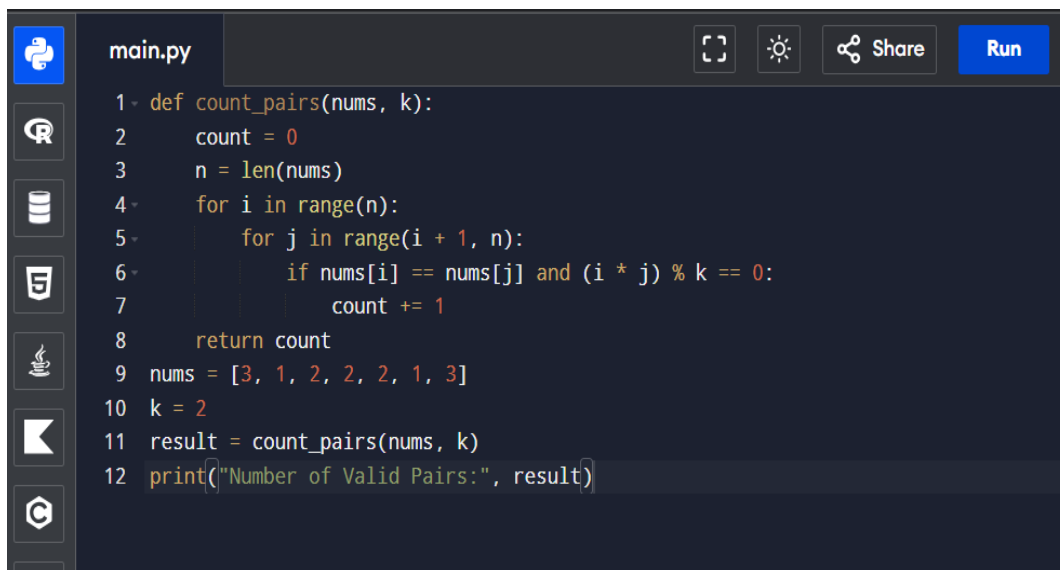
Step 7: If both conditions are true, increment count by 1.

Step 8: Repeat until all pairs are checked.

Step 9: Display the total number of valid pairs.

Step 10: Stop the program.

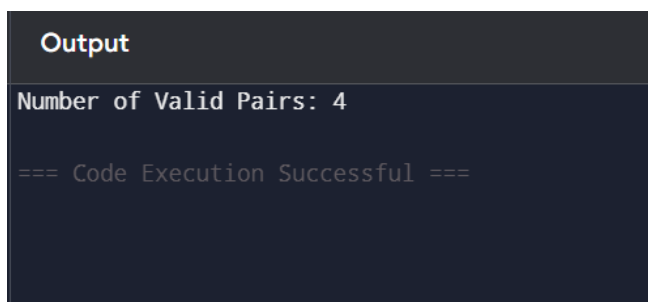
Source code

A screenshot of a code editor interface. The editor has a dark theme. On the left, there is a sidebar with icons for Python, a debugger, a file explorer, a terminal, a search bar, and a refresh button. The main area shows a file named 'main.py' with the following Python code:

```
1 def count_pairs(nums, k):
2     count = 0
3     n = len(nums)
4     for i in range(n):
5         for j in range(i + 1, n):
6             if nums[i] == nums[j] and (i * j) % k == 0:
7                 count += 1
8     return count
9 nums = [3, 1, 2, 2, 2, 1, 3]
10 k = 2
11 result = count_pairs(nums, k)
12 print("Number of Valid Pairs:", result)
```

At the top right of the editor, there are icons for a full-screen view, a settings gear, a share button, and a blue 'Run' button.

Output

A screenshot of the output window of the code editor. It has a dark background with the text 'Output' in white at the top. Below it, the output of the program is displayed in white text:

```
Number of Valid Pairs: 4
```

At the bottom, there is a green message indicating successful execution:

```
=== Code Execution Successful ===
```

Time Complexity

Let **n** be the number of elements in the array.

- The outer loop runs **n** times.
- The inner loop runs approximately **n** times.

Therefore,

$$\text{Time Complexity} = O(n^2)$$

Space Complexity

The algorithm uses only a few extra variables (count, i, j).

No additional data structures are used.

$$\boxed{\text{Space Complexity} = O(1)}$$

Result

Thus, the Python program to count the number of equal and divisible index pairs was successfully implemented and executed.

Experiment No: 5

Experiment Name

Find the Maximum Element in an Array with the Least Time Complexity

Aim

To develop a Python program to find the maximum element in a given array using the least possible time complexity by traversing the array only once.

Algorithm

Step 1: Start the program.

Step 2: Read the number of elements and the array.

Step 3: Assume the first element is the maximum.

Step 4: Traverse the remaining elements of the array.

Step 5: Compare each element with the current maximum.

Step 6: If the current element is greater than the maximum, update the maximum value.

Step 7: Repeat until all elements are checked.

Step 8: Display the maximum element.

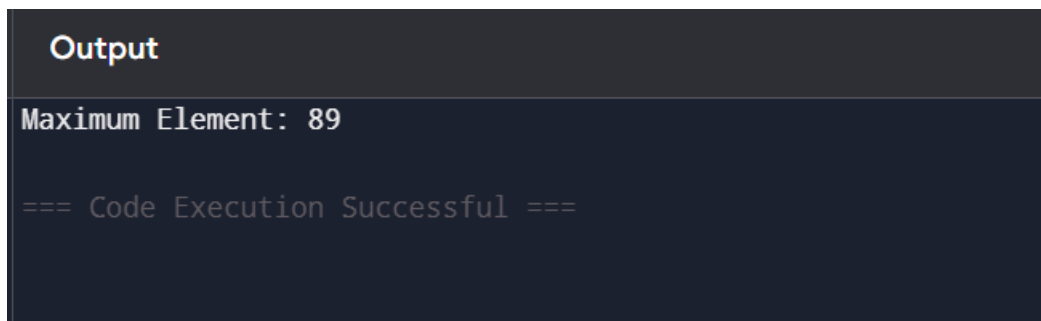
Step 9: Stop the program.

Source code



```
main.py
1 def find_maximum(arr):
2     maximum = arr[0]
3     for i in range(1, len(arr)):
4         if arr[i] > maximum:
5             maximum = arr[i]
6     return maximum
7 arr = [12, 45, 7, 89, 23, 56]
8 result = find_maximum(arr)
9 print("Maximum Element:", result)
```

Output



```
Output
Maximum Element: 89

=== Code Execution Successful ===
```

Time Complexity

Let n be the number of elements in the array.

- The array is traversed only once.
- Each element is compared exactly one time.

Therefore,

$$\text{Time Complexity} = O(n)$$

This is the minimum possible time complexity for finding the maximum element in an unsorted array.

Space Complexity

The algorithm uses only one extra variable (maximum) to store the current largest value.

Therefore,

$$\text{Space Complexity} = O(1)$$

Result

Thus, the Python program to find the maximum element in an array using the least time complexity was successfully implemented and executed.

Experiment No 6

Experiment Name

Sort the List Using an Efficient Sorting Algorithm and Find the Maximum Element

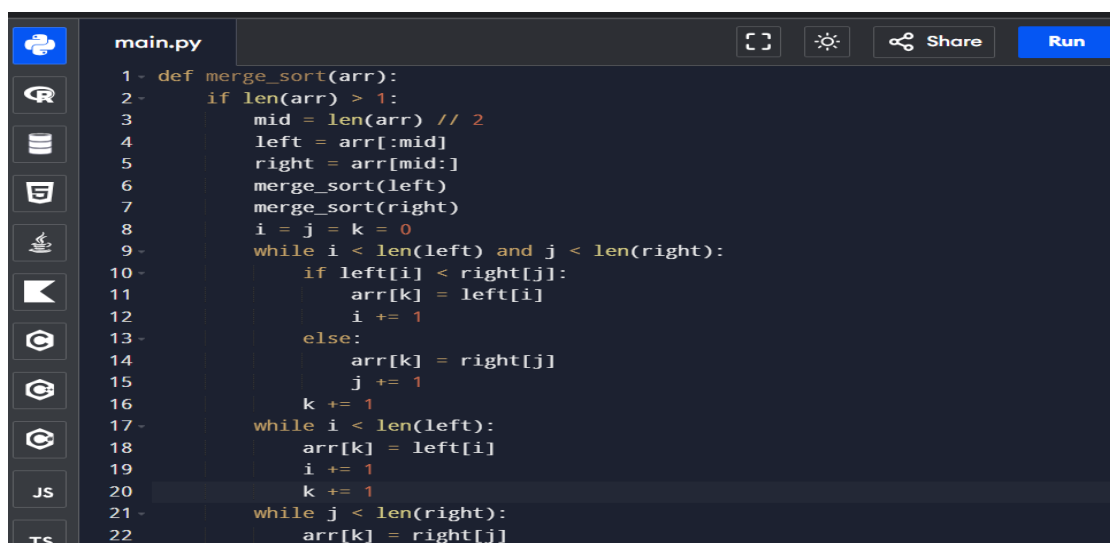
Aim

To develop a program that sorts a list of numbers using Merge Sort (an efficient sorting algorithm) and then finds the maximum element from the sorted list.

Algorithm

1. Start.
2. Read the list of numbers.
3. Check whether the list is empty.
 - If empty, display an appropriate message and stop.
4. Apply Merge Sort to sort the list in ascending order.
5. After sorting, access the last element of the list.
6. Print the sorted list.
7. Print the maximum element.
8. Stop.

SOURCE CODE



```
main.py
1 - def merge_sort(arr):
2 -     if len(arr) > 1:
3 -         mid = len(arr) // 2
4 -         left = arr[:mid]
5 -         right = arr[mid:]
6 -         merge_sort(left)
7 -         merge_sort(right)
8 -         i = j = k = 0
9 -         while i < len(left) and j < len(right):
10 -             if left[i] < right[j]:
11 -                 arr[k] = left[i]
12 -                 i += 1
13 -             else:
14 -                 arr[k] = right[j]
15 -                 j += 1
16 -             k += 1
17 -         while i < len(left):
18 -             arr[k] = left[i]
19 -             i += 1
20 -             k += 1
21 -         while j < len(right):
22 -             arr[k] = right[j]
```

```

        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
numbers = list(map(int, input("Enter the numbers: ").split()))
if len(numbers) == 0:
    print("The list is empty.")
else:
    merge_sort(numbers)
    print("Sorted List :", numbers)
    print("Maximum Element :", numbers[-1])

```

OUTPUT

```

Output
Enter the numbers: 5
Sorted List : [5]
Maximum Element : 5

=== Code Execution Successful ===

```

Time Complexity

- Merge Sort: $O(n \log n)$
- Finding Maximum Element: $O(1)$
- Overall Time Complexity: $O(n \log n)$

Space Complexity

- Merge Sort requires additional memory for temporary arrays.

Overall Space Complexity: $O(n)$

Result

Thus, the program was successfully implemented to sort a list using the Merge Sort algorithm and find the maximum element from the sorted list.

Experiment No 7

Experiment Name

Create a List of Unique Elements from a Given List and Analyze Its Space Complexity

Aim

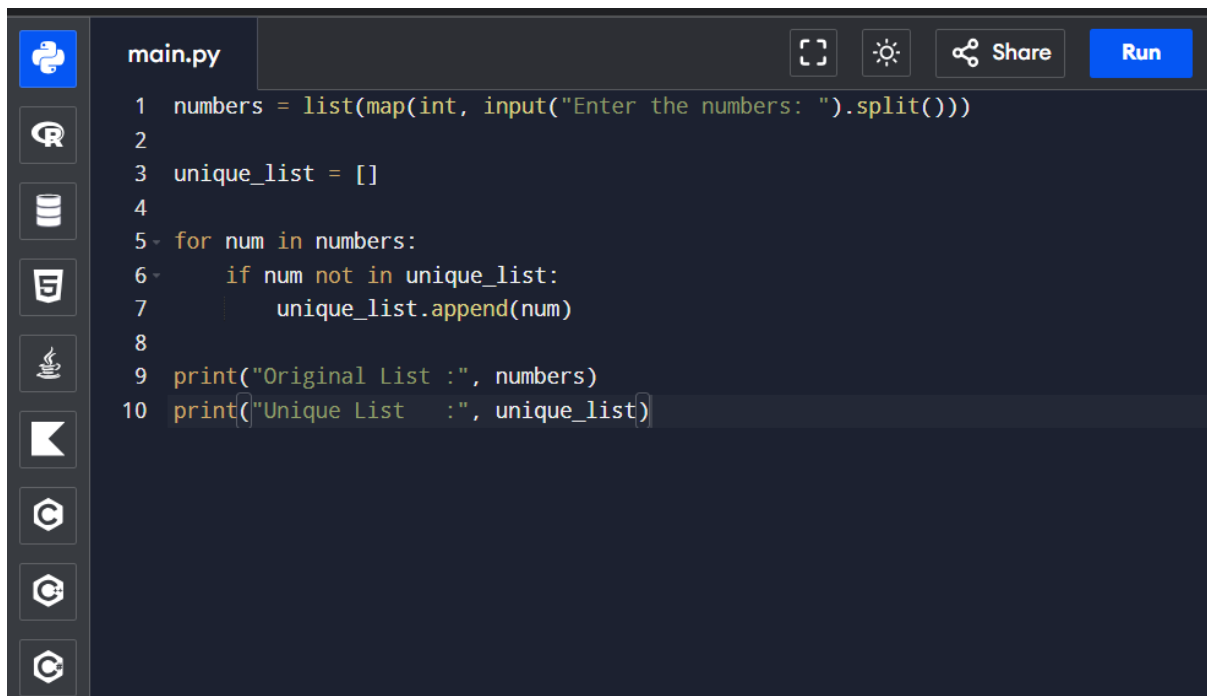
To develop a program that removes duplicate elements from a given list and creates a new list containing only unique elements while preserving the order of their first occurrence.

Algorithm

1. Start the program.

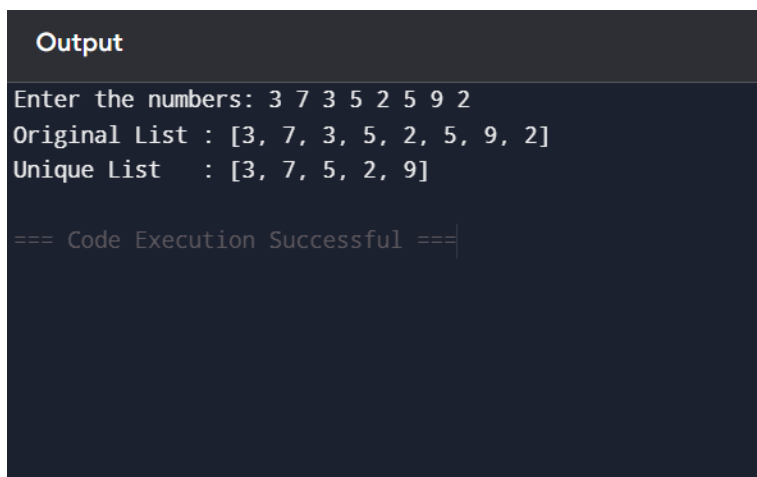
2. Read the list of numbers from the user.
3. Create an empty list called `unique_list`.
4. Traverse each element in the input list.
5. Check whether the element already exists in `unique_list`.
 - If it does not exist, add it to `unique_list`.
 - Otherwise, ignore it.
6. Display the list of unique elements.
7. Stop the program.

Source Code



```
main.py
1 numbers = list(map(int, input("Enter the numbers: ").split()))
2
3 unique_list = []
4
5 for num in numbers:
6     if num not in unique_list:
7         unique_list.append(num)
8
9 print("Original List :", numbers)
10 print("Unique List   :", unique_list)
```

OUTPUT



```
Output
Enter the numbers: 3 7 3 5 2 5 9 2
Original List : [3, 7, 3, 5, 2, 5, 9, 2]
Unique List   : [3, 7, 5, 2, 9]

=== Code Execution Successful ===
```

Time Complexity

- Traversing the list: $O(n)$
- Checking duplicates using a list (num not in unique_list): $O(n)$

Overall Time Complexity: $O(n^2)$

Note: If a set is used for duplicate checking, the time complexity can be improved to $O(n)$.

Space Complexity

- The program stores the unique elements in a separate list.

Overall Space Complexity: $O(n)$

Result

Thus, the program was successfully implemented to create a new list containing only the unique elements from the given input list.

Experiment No 8

Experiment Name

Bubble Sort Algorithm

Aim

To implement the Bubble Sort algorithm for sorting an array of integers in ascending order and analyse its time complexity and space complexity.

Algorithm

Step 1: Start.

Step 2: Read the number of elements n .

Step 3: Read the array elements.

Step 4: Repeat for $i = 0$ to $n-2$

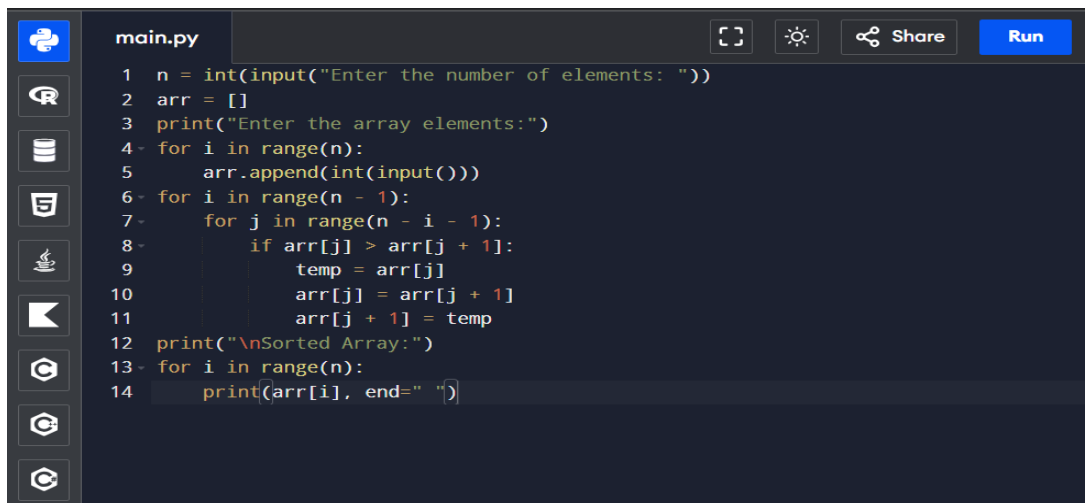
- Repeat for $j = 0$ to $n-i-2$
 - If $arr[j] > arr[j+1]$
 - Swap the two elements.

Step 5: Repeat until no more swaps are required.

Step 6: Display the sorted array.

Step 7: Stop.

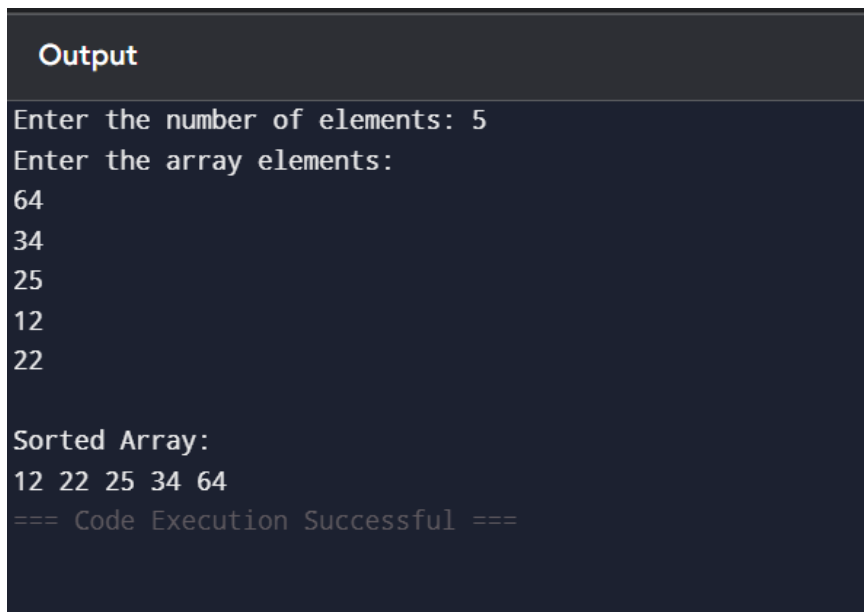
Source Code



```
main.py
1 n = int(input("Enter the number of elements: "))
2 arr = []
3 print("Enter the array elements:")
4 for i in range(n):
5     arr.append(int(input()))
6 for i in range(n - 1):
7     for j in range(n - i - 1):
8         if arr[j] > arr[j + 1]:
9             temp = arr[j]
10            arr[j] = arr[j + 1]
11            arr[j + 1] = temp
12 print("\nSorted Array:")
13 for i in range(n):
14     print(arr[i], end=" ")

Run
```

Output



```
Output
Enter the number of elements: 5
Enter the array elements:
64
34
25
12
22

Sorted Array:
12 22 25 34 64
=== Code Execution Successful ===
```

Time Complexity

Case	Complexity
Best Case (Already Sorted)	$O(n)$
Average Case	$O(n^2)$
Worst Case (Reverse Sorted)	$O(n^2)$

Space Complexity

$O(1)$

Bubble Sort is an in-place sorting algorithm because it requires only one temporary variable for swapping.

Result

The Bubble Sort algorithm was successfully implemented in C. The given array was sorted in ascending order.

Experiment No 9

Experiment Name

Binary Search Algorithm

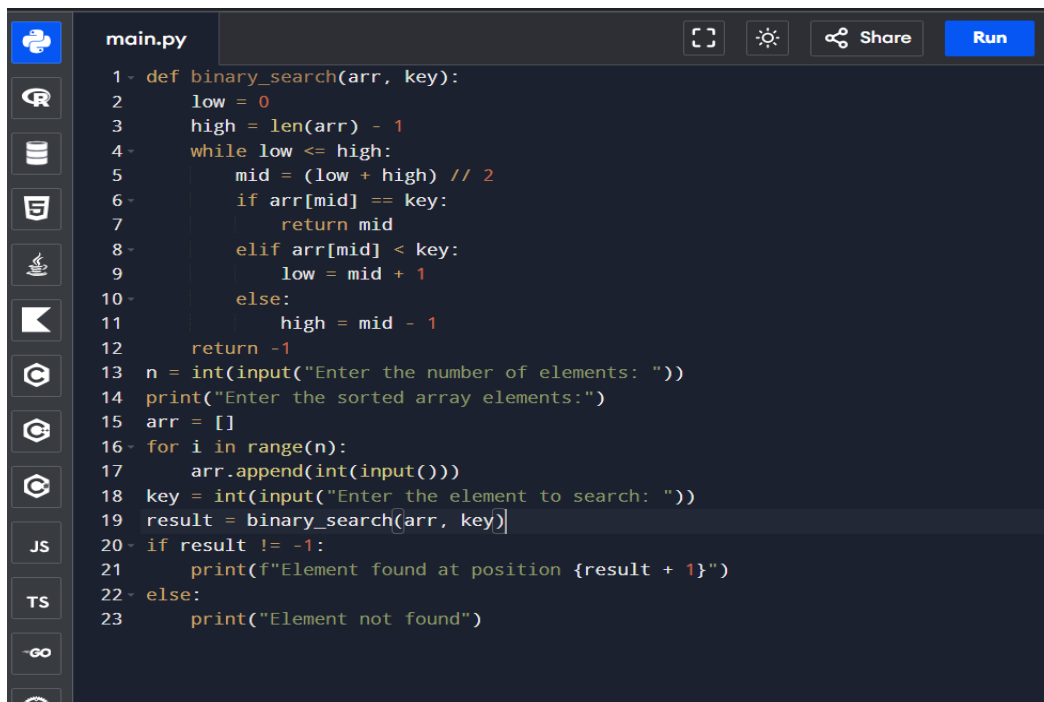
Aim

To implement the Binary Search algorithm to search for a given element in a sorted array and analyse its time and space complexity.

Algorithm

1. Start.
2. Read the number of elements n .
3. Read the sorted array elements.
4. Read the search key.
5. Initialize $low = 0$ and $high = n - 1$.
6. Repeat while $low \leq high$:
 - Find $mid = (low + high) / 2$.
 - If $arr[mid] == key$, display the position and stop.
 - If $key < arr[mid]$, set $high = mid - 1$.
 - Otherwise, set $low = mid + 1$.
7. If the element is not found, display an appropriate message.
8. Stop.

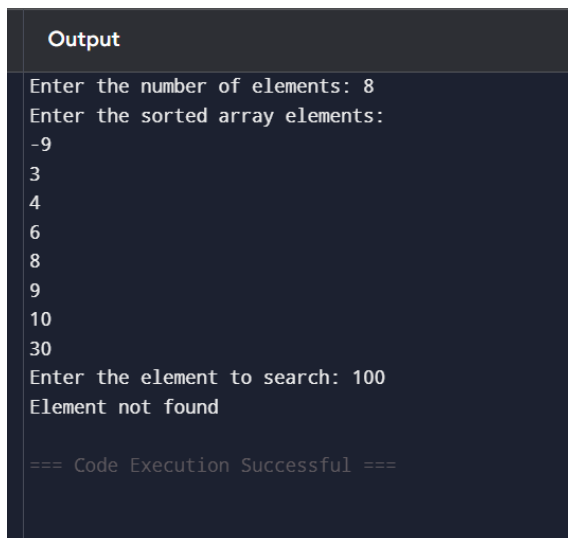
Source Code



The screenshot shows a code editor with a dark theme. The file is named 'main.py'. The code implements a binary search function and a main program that takes user input. The code is as follows:

```
1- def binary_search(arr, key):
2-     low = 0
3-     high = len(arr) - 1
4-     while low <= high:
5-         mid = (low + high) // 2
6-         if arr[mid] == key:
7-             return mid
8-         elif arr[mid] < key:
9-             low = mid + 1
10-        else:
11-            high = mid - 1
12-    return -1
13- n = int(input("Enter the number of elements: "))
14- print("Enter the sorted array elements:")
15- arr = []
16- for i in range(n):
17-     arr.append(int(input()))
18- key = int(input("Enter the element to search: "))
19- result = binary_search(arr, key)
20- if result != -1:
21-     print(f"Element found at position {result + 1}")
22- else:
23-     print("Element not found")
```

Output



The screenshot shows the output of the program in a dark-themed window. The output is as follows:

```
Output
Enter the number of elements: 8
Enter the sorted array elements:
-9
3
4
6
8
9
10
30
Enter the element to search: 100
Element not found

=== Code Execution Successful ===
```

Time Complexity

Case	Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Space Complexity

O (1) (Iterative implementation)

Result

The Binary Search algorithm was successfully implemented in C.

Experiment No 10

Experiment Name

Merge Sort Algorithm

Aim

To implement the Merge Sort algorithm to sort an array of integers in ascending order without using built-in sorting functions and analyse its time and space complexity.

Algorithm

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. Divide the array into two equal halves.
5. Recursively sort the left half.
6. Recursively sort the right half.
7. Merge the two sorted halves into a single sorted array.
8. Display the sorted array.
9. Stop.

Source Code

```

main.py
1 def merge(arr, left, mid, right):
2     left_part = arr[left:mid + 1]
3     right_part = arr[mid + 1:right + 1]
4     i = j = 0
5     k = left
6     while i < len(left_part) and j < len(right_part):
7         if left_part[i] <= right_part[j]:
8             arr[k] = left_part[i]
9             i += 1
10        else:
11            arr[k] = right_part[j]
12            j += 1
13        k += 1
14    while i < len(left_part):
15        arr[k] = left_part[i]
16        i += 1
17        k += 1
18    while j < len(right_part):
19        arr[k] = right_part[j]
20        j += 1
21        k += 1
22 def merge_sort(arr, left, right):
23     if left < right:
24         mid = (left + right) // 2
25         merge_sort(arr, left, mid)
26         merge_sort(arr, mid + 1, right)
27         merge(arr, left, mid, right)
28 n = int(input("Enter the number of elements: "))
29 arr = []

j += 1
k += 1
while i < len(left_part):
    arr[k] = left_part[i]
    i += 1
    k += 1
while j < len(right_part):
    arr[k] = right_part[j]
    j += 1
    k += 1
def merge_sort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid)
        merge_sort(arr, mid + 1, right)
        merge(arr, left, mid, right)
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the array elements:")
for i in range(n):
    arr.append(int(input()))
merge_sort(arr, 0, n - 1)
print("\nSorted Array:")

```

Output

```

Output
Enter the number of elements: 8
Enter the array elements:
12
45
7
89
23
56
1
34

Sorted Array:
1 7 12 23 34 45 56 89

=== Code Execution Successful ===

```

Time Complexity

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Space Complexity

$O(n)$ (Additional temporary arrays are used during merging.)

Result

The Merge Sort algorithm was successfully implemented in C.

Experiment No 11

Experiment Name

Out of Boundary Paths (Dynamic Programming)

Aim

To find the number of ways a ball can move out of an $m \times n$ grid boundary in exactly N moves using Dynamic Programming.

Algorithm

1. Read the values of m, n, N, i, j .
2. Create a DP table to store the number of ways to reach each cell.
3. Initialize the starting position with 1.
4. For each move from **1 to N**:
 - Move in four directions (up, down, left, right).
 - If the move goes outside the grid, increment the answer.
 - Otherwise, update the next DP table.
5. Print the total number of paths.

Source Code

```

1- def findPaths(m, n, N, i, j):
2     MOD = 10**9 + 7
3     dp = [[0] * n for _ in range(m)]
4     dp[i][j] = 1
5     ans = 0
6
7     directions = [(1,0),(-1,0),(0,1),(0,-1)]
8
9     for _ in range(N):
10        temp = [[0] * n for _ in range(m)]
11
12        for r in range(m):
13            for c in range(n):
14                if dp[r][c]:
15                    for dr, dc in directions:
16                        nr, nc = r + dr, c + dc
17
18                        if nr < 0 or nr >= m or nc < 0 or nc >= n:
19                            ans = (ans + dp[r][c]) % MOD
20                        else:
21                            temp[nr][nc] = (temp[nr][nc] + dp[r][c]) % MOD
22        dp = temp
23
24    return ans
25
26    m = 2
27    n = 2
28    N = 2
29    i = 0
30    j = 0
31
32    print(findPaths(m, n, N, i, j))

```

Output

```

Output
6

=== Code Execution Successful ===

```

Time Complexity:

$O(N \times m \times n)$

Space Complexity:

$O(m \times n)$

Result

The program successfully calculates the number of paths by which the ball moves outside the grid in exactly N moves.

Experiment No 12

Experiment Name

House Robber II (Dynamic Programming)

Aim

To find the maximum amount of money that can be robbed from houses arranged in a circular street without robbing two adjacent houses.

Algorithm

1. Read the array of house values.
2. If there is only one house, return its value.
3. Since the houses are circular:
 - Rob houses from **0 to n-2**.
 - Rob houses from **1 to n-1**.
4. For each case, use Dynamic Programming:
 - Keep track of the maximum amount robbed without selecting adjacent houses.
5. Return the maximum of the two cases.

Source Code

```
1 def robLinear(nums):
2     prev1 = 0
3     prev2 = 0
4
5     for money in nums:
6         temp = max(prev1, prev2 + money)
7         prev2 = prev1
8         prev1 = temp
9
10    return prev1
11
12
13 def rob(nums):
14     if len(nums) == 1:
15         return nums[0]
16
17     return max(robLinear(nums[:-1]), robLinear(nums[1:]))
18
19 nums = [1, 2, 3, 1]
20 print(rob(nums))
```

Output

Output

4

=== Code Execution Successful ===

Time Complexity: $O(n)$

$O(n) + O(n) = O(n)$

Space Complexity: $O(1)$

$O(1)$

Result

The program successfully finds the maximum amount of money that can be robbed without robbing adjacent houses in a circular arrangement.

Experiment No 13

Experiment Name

Climbing Stairs (Dynamic Programming)

Aim

To find the number of distinct ways to climb a staircase when each step can be either 1 or 2 steps at a time.

Algorithm

1. Read the number of stairs n .
2. If $n = 1$, return 1.
3. Initialize two variables:
 - $first = 1$
 - $second = 2$
4. From step 3 to n , calculate:
 - $current = first + second$
 - Update first and second.
5. Print the total number of ways.

Source code

```
1 def climbStairs(n):
2     if n <= 2:
3         return n
4
5     first = 1
6     second = 2
7
8     for i in range(3, n + 1):
9         current = first + second
10        first = second
11        second = current
12
13    return second
14
15 n = int(input("Enter number of stairs: "))
16 print("Number of ways:", climbStairs(n))
```

Output

```
Output
Enter number of stairs: 4
Number of ways: 5

=== Code Execution Successful ===
```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

Result

The program successfully calculates the number of distinct ways to climb the staircase.

Experiment No 14

Experiment Name

Unique Paths

Aim

To find the number of unique paths from the top-left corner to the bottom-right corner of an $m \times n$ grid.

Algorithm

1. Read the values of m and n .
2. Create a DP table of size $m \times n$.
3. Fill the first row and first column with 1.
4. For every remaining cell:
 - $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
5. Print the value at $dp[m-1][n-1]$.

Source code

```
1 def uniquePaths(m, n):
2     dp = [[1] * n for _ in range(m)]
3
4     for i in range(1, m):
5         for j in range(1, n):
6             dp[i][j] = dp[i-1][j] + dp[i][j-1]
7
8     return dp[m-1][n-1]
9
10 m = int(input("Enter rows: "))
11 n = int(input("Enter columns: "))
12
13 print("Unique Paths:", uniquePaths(m, n))
```

Output

```
Output
Enter rows: 7
Enter columns: 3
Unique Paths: 28

=== Code Execution Successful ===
```

Time Complexity:

$O(m \times n)$

Space Complexity:

$O(m \times n)$

Result

The program successfully finds the number of unique paths in the grid.

Experiment No 15

Experiment Name

Positions of Large Groups (String Processing)

Aim

To find all large groups ($\text{length} \geq 3$) in a string and return their starting and ending indices.

Algorithm

1. Read the input string.
2. Initialize an empty list for the result.
3. Traverse the string using two pointers.
4. Count consecutive identical characters.
5. If the group length is **3 or more**, store its start and end indices.
6. Print the list of intervals.

Source code

```
1 def largeGroupPositions(s):
2     result = []
3     i = 0
4
5     while i < len(s):
6         j = i
7         while j < len(s) and s[j] == s[i]:
8             j += 1
9
10        if j - i >= 3:
11            result.append([i, j - 1])
12
13        i = j
14
15    return result
16
17 s = input("Enter string: ")
18 print(largeGroupPositions(s))
```

Output

```
Output
Enter string: abbxxxxzy
[[3, 6]]

=== Code Execution Successful ===
```

Time Complexity:

$O(n)$

Space Complexity:

$O(k)$

Result

The program successfully identifies all large groups and prints their starting and ending indices.

Experiment No 16

Experiment Name

Game of Life (Cellular Automaton)

Aim

To simulate the next state of a 2D grid based on the Game of Life rules using Cellular Automata.

Algorithm

1. Read the input grid (board).
2. For each cell, count the number of live neighbours.
3. Apply the Game of Life rules:
 - A live cell with fewer than 2 live neighbours dies.
 - A live cell with 2 or 3 live neighbours lives.
 - A live cell with more than 3 live neighbours dies.
 - A dead cell with exactly 3 live neighbours becomes alive.
4. Store the updated state in a new grid.
5. Copy the new grid to the original board.
6. Print the updated board.

Source code

```

1 def gameOfLife(board):
2     rows = len(board)
3     cols = len(board[0])
4
5     directions = [(-1,-1),(-1,0),(-1,1),
6                   (0,-1),(0,1),
7                   (1,-1),(1,0),(1,1)]
8
9     new_board = [[0] * cols for _ in range(rows)]
10
11     for i in range(rows):
12         for j in range(cols):
13             live = 0
14
15             for dx, dy in directions:
16                 ni = i + dx
17                 nj = j + dy
18
19                 if 0 <= ni < rows and 0 <= nj < cols:
20                     live += board[ni][nj]
21
22                 if board[i][j] == 1:
23                     if live == 2 or live == 3:
24                         new_board[i][j] = 1
25                 else:
26                     if live == 3:
27                         new_board[i][j] = 1
28
29     return new_board
30
31 board = [[0,1,0],
32          [0,0,1],

```

Output

```

Output
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]

=== Code Execution Successful ===

```

Time Complexity:

$O(m \times n)$

Space Complexity:

$O(m \times n)$

Result

The program successfully computes and displays the next generation of the Game of Life according to the given rules.

Experiment No 17

Experiment Name

Champagne Tower

Aim

To determine the amount of champagne contained in a specified glass after pouring a given number of cups into the top glass of a champagne tower.

Algorithm

1. Read the values of poured, query_row, and query_glass.
2. Create a DP table of size 102×102 initialized to 0.
3. Pour the given number of cups into the top glass ($dp[0][0]$).
4. Traverse each row:
 - If a glass contains more than 1 cup, calculate the excess.
 - Distribute half of the excess to the left child and half to the right child.
5. Continue until the required row is processed.
6. Return the minimum of 1 and the value in the queried glass.

Source code

```
1 def champagneTower(poured, query_row, query_glass):
2     dp = [[0.0] * 102 for _ in range(102)]
3     dp[0][0] = poured
4
5     for row in range(query_row + 1):
6         for col in range(row + 1):
7             excess = (dp[row][col] - 1.0) / 2.0
8
9             if excess > 0:
10                dp[row + 1][col] += excess
11                dp[row + 1][col + 1] += excess
12
13     return min(1, dp[query_row][query_glass])
14
15 poured = int(input("Enter poured cups: "))
16 query_row = int(input("Enter query row: "))
17 query_glass = int(input("Enter query glass: "))
18
19 print("Amount of champagne:", champagneTower(poured, query_row, query_glass))
```

Output

Output

```
Enter poured cups: 2  
Enter query row: 1  
Enter query glass: 1  
Amount of champagne: 0.5
```

```
=== Code Execution Successful ===
```

Time Complexity:

$O(\text{query_row}^2)$

Space Complexity:

$O(\text{query_row}^2)$

Result

The program successfully computes the amount of champagne present in the specified glass after pouring the given number of cups.